🚀 DAY 24: JAVASCRIPT EVENT LOOP & MICROTASKS

Lakshmiprasanna Vakada

Have you ever wondered how JavaScript handles multiple tasks efficiently, even though it's single-threaded? 🤔

The secret lies in the Event Loop, which manages the execution of synchronous and asynchronous tasks. Let’s break it down! 👇

🔹 What is the JavaScript Event Loop?

JavaScript is single-threaded, meaning it executes code one line at a time in the call stack. But what if we have an async operation like:

- ✅ Fetching API data
- ✅ Handling user interactions
- ✅ Running setTimeout

This is where the Event Loop steps in to manage asynchronous tasks without blocking the main thread. 🚀

🔹 JavaScript Execution Flow

JavaScript executes code in the following order:

1️⃣ Call Stack → Runs synchronous code (LIFO: Last In, First Out)
2️⃣ Web APIs → Handles async tasks like setTimeout, DOM events, fetch, etc.
3️⃣ Callback Queue → Stores setTimeout & setInterval callbacks
4️⃣ Microtask Queue → Stores Promises & MutationObserver callbacks (higher priority than Callback Queue)
5️⃣ Event Loop → Moves tasks from queues to the Call Stack when it’s empty

🔷 Understanding the Call Stack

📌 The Call Stack is where JavaScript executes functions in LIFO order.

```
function first() {
  console.log("First function");
}
function second() {
  console.log("Second function");
}
first();
second();
console.log("End of script");
```

🔷 Output:

```
First function
Second function
End of script
```

✅ Since all functions are synchronous, they execute immediately in the Call Stack.

How Asynchronous Tasks Work? (setTimeout Example)

```
console.log("Start");
setTimeout(() => {
  console.log("Inside setTimeout");
}, 0);
console.log("End");
```

- **Output:**

```
Start
End
Inside setTimeout
```

✅ Even though setTimeout is oms, it doesn't run immediately because it moves to the Callback Queue, and JavaScript first finishes executing synchronous code.

🔹 Microtask Queue vs Callback Queue

The Microtask Queue has higher priority than the Callback Queue.

📌 Microtasks:

- Promises (.then() & .catch())
- MutationObserver

📌 Callback Queue:

- setTimeout
- setInterval

Example: Microtasks vs Callback Queue

```
console.log("Start");
setTimeout(() => {
  console.log("setTimeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise resolved");
});
console.log("End");
```

- Output:

```
Start
End
Promise resolved
setTimeout
```

✅ Even though setTimeout has oms, the Promise executes first because Microtasks always run before Callback Queue.

🔹 Event Loop in Action

1️⃣ Synchronous code runs first (Call Stack)
2️⃣ Microtasks (Promises) execute next
3️⃣ Callbacks (setTimeout, setInterval) run last

Example: Event Loop Execution

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() =>
console.log("3"));
console.log("4");
```

🔹 Execution Order:

```
1
4
3  ✅ (Microtask runs before setTimeout)
2
```